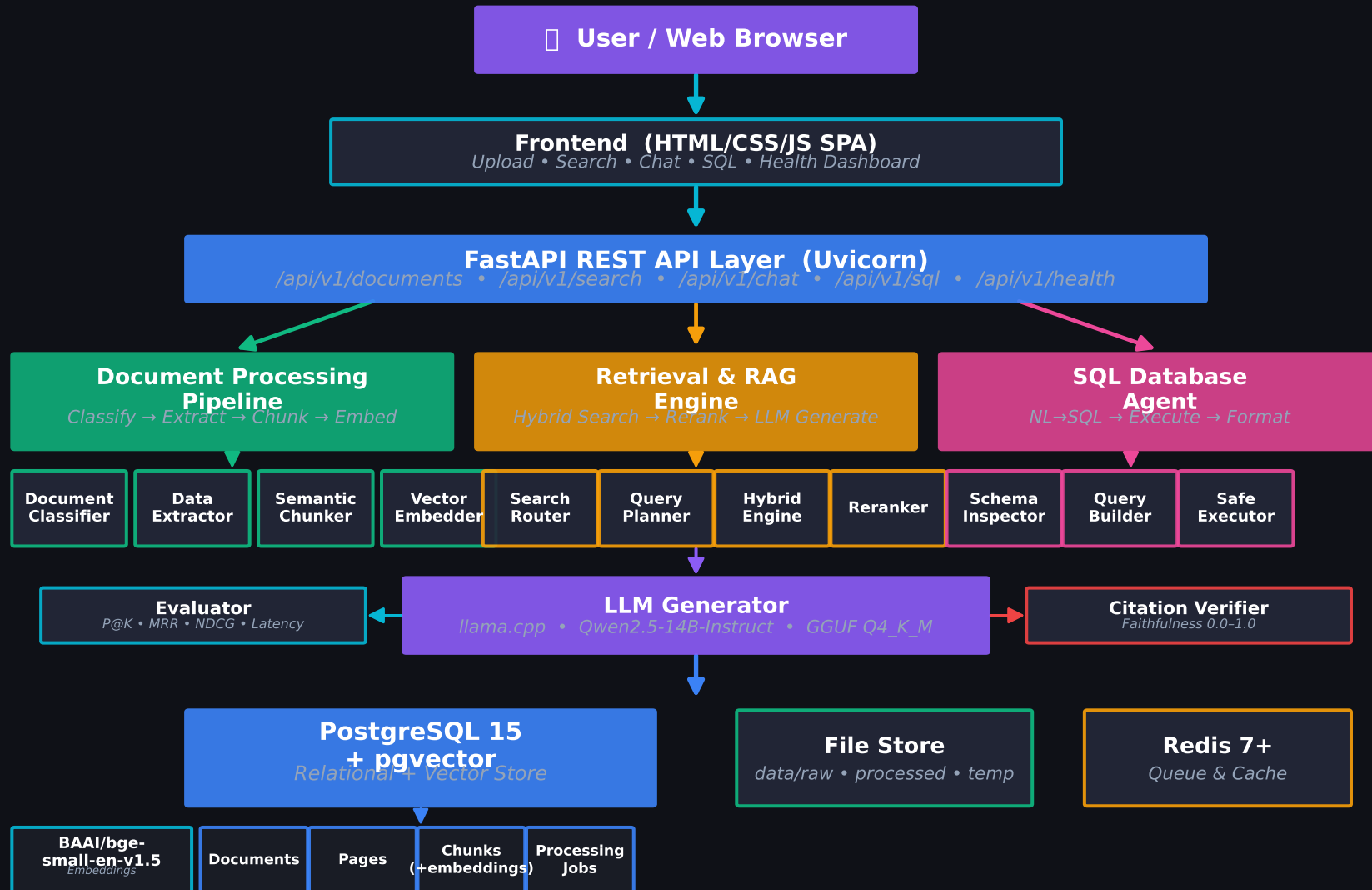


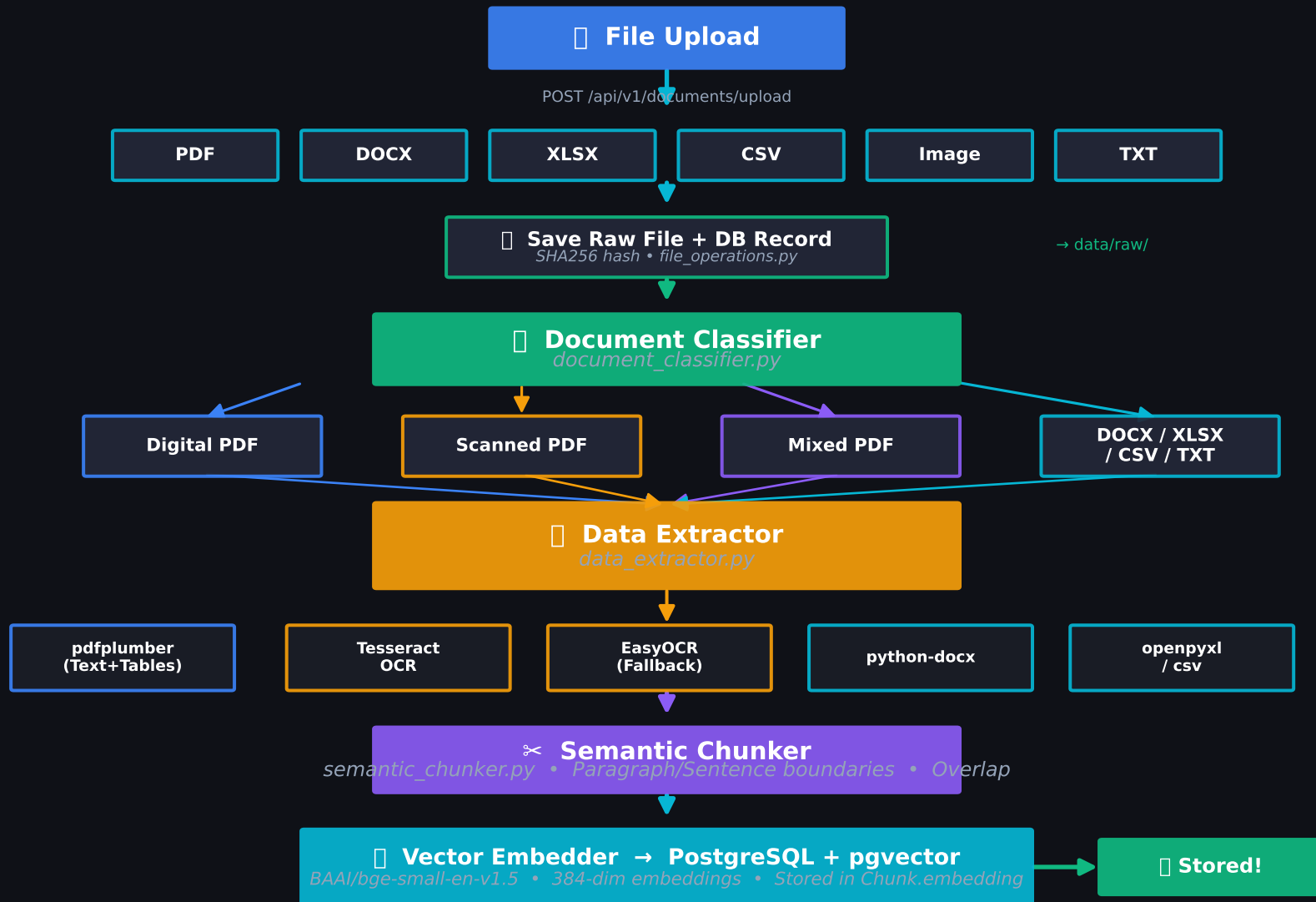
DocuRAG — High-Level System Architecture

Enterprise AI Document Intelligence & RAG System



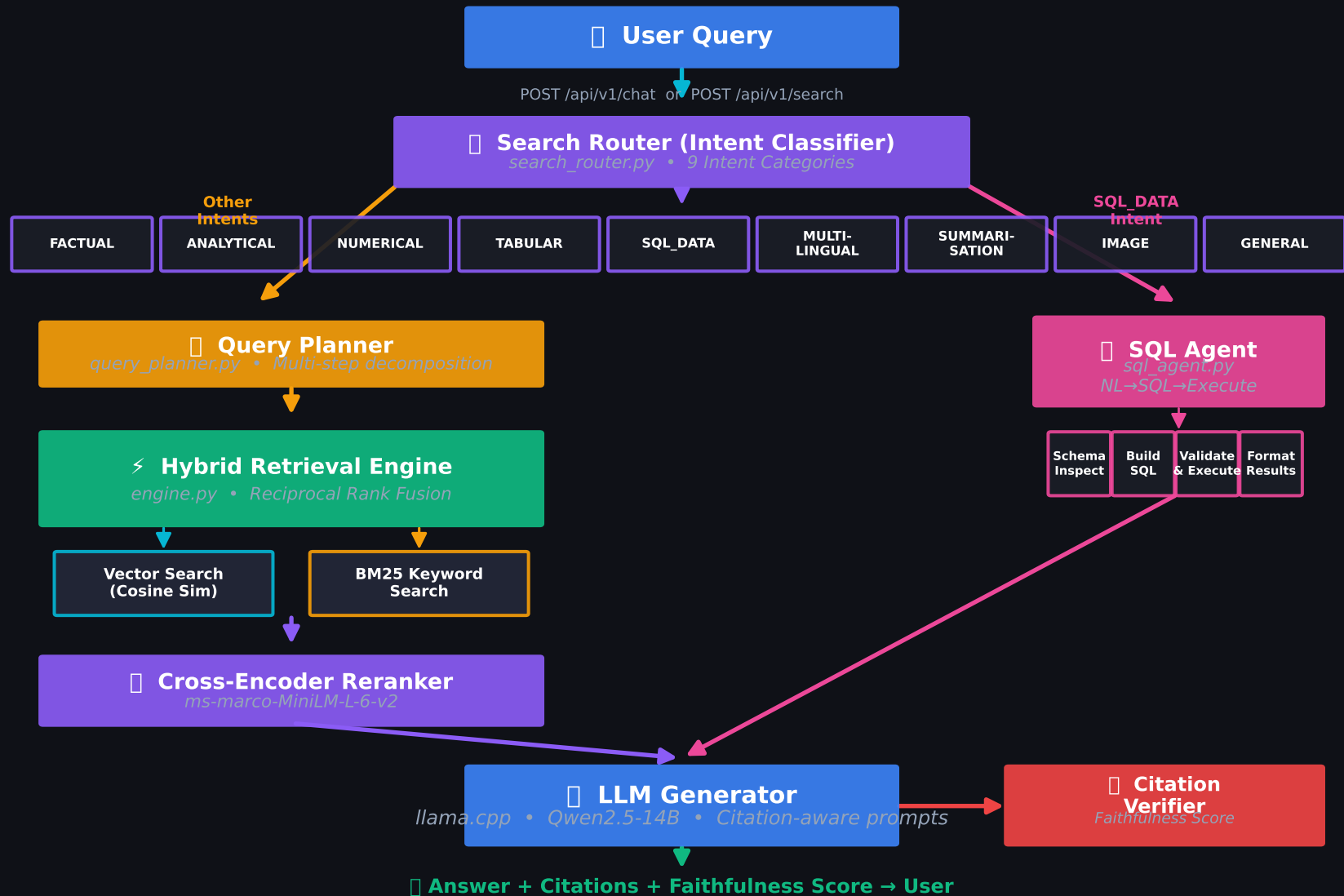
Document Ingestion Pipeline

Phase 1-4: Upload → Classify → Extract → Chunk → Embed → Store



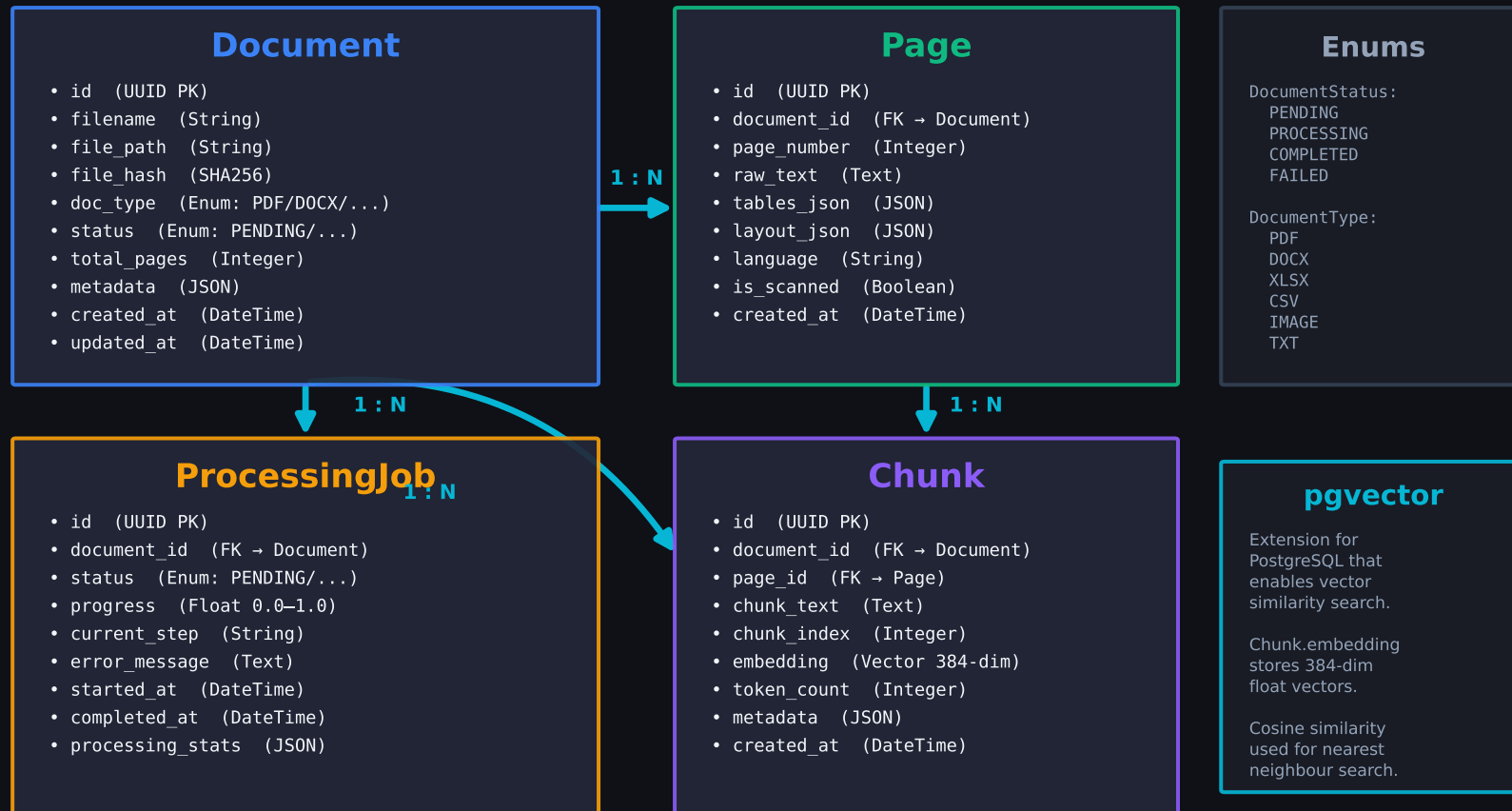
Retrieval & RAG Generation Pipeline

Phase 5-9: Query → Route → Retrieve → Rerank → Generate → Cite



Database Models & Data Relationships

SQLAlchemy 2.0 (Async) + PostgreSQL 15 + pgvector



All models defined in `src/database_models/` • Migrations via Alembic • Async sessions via SQLAlchemy 2.0

API Layer & Frontend Architecture

FastAPI REST Endpoints + Single-Page Application

FastAPI REST API — /api/v1/

/health

GET /health → System status

/documents

POST /upload → Ingest file
GET / → List all docs
GET /{id} → Doc details
DELETE /{id} → Remove doc

/search

POST / → Hybrid search
GET /stats → Index stats

/chat

POST / → RAG-powered chat

/sql

POST /connect → Register DB
DELETE /connect/{label} → Deregister
GET /connections → List DBs
GET /schema/{label} → Inspect
POST /query → NL-SQL

/eval

POST /run → Quality benchmarks
GET /results → Eval metrics

JSON Request/Response • CORS enabled • Swagger /docs • ReDoc /redoc

↓ HTTP / JSON

Frontend — Single Page Application

📁 Upload

Drag & drop
file upload
progress bar

🔍 Search

Hybrid search
results with
citations

💬 Chat

Conversational
RAG with
faithfulness

🗄️ SQL

Connect DBs
NL query
schema inspect

❤️ Health

System metrics
CPU/Memory
model status

index.html + css/index.css + js/app.js

Dark Theme • Glassmorphism • Responsive • Served as Static Files by FastAPI

Project File Tree & Module Map

Complete directory structure with module responsibilities

```

estate/ (DocuRAG Root)
├─ application_configuration/
│   ├── environment_settings.py    Pydantic Settings (.env)
│   └── logger_setup.py           Structlog / logging config
├─ src/
│   ├── main_application.py        FastAPI app entrypoint
│   └─ api/
│       ├── v1_router.py          API v1 router aggregator
│       └─ routes/
│           ├── health_routes.py   GET /health
│           ├── document_routes.py  CRUD + upload endpoints
│           ├── search_routes.py    Hybrid search endpoints
│           ├── chat_routes.py      RAG chat endpoint
│           └── sql_routes.py       SQL agent endpoints
│
│   ├── database_models/
│       ├── database_connection.py  Async engine + sessions
│       ├── shared_enums.py         Status & Type enums
│       ├── document_model.py       Document ORM model
│       ├── page_model.py           Page ORM model
│       ├── chunk_model.py          Chunk + embedding ORM
│       └── processing_job_model.py  Job tracking ORM
│
│   ├── document_processing/
│       ├── ingestion_pipeline.py    Orchestrator
│       ├── document_classifier.py   Digital/Scanned/Mixed
│       ├── data_extractor.py        Multi-format extraction
│       ├── semantic_chunker.py      Boundary-aware chunking
│       ├── vector_embedder.py       BGE embedding + store
│       └── processing_schemas.py    Pydantic DTOs
│
│   ├── retrieval/
│       ├── engine.py                Hybrid retrieval + RRF
│       ├── retriever.py             Vector + BM25 retrievers
│       ├── reranker.py              Cross-encoder reranker
│       ├── search_router.py         9-intent classifier
│       ├── query_planner.py         Multi-step decomposition
│       ├── search_service.py        High-level orchestrator
│       ├── sql_agent.py             NL-SQL agent
│       ├── citation_verifier.py     Faithfulness scoring
│       └── evaluator.py             P@K, MRR, NDCG metrics
│
│   ├── llm/
│       └── generator.py             llama.cpp + extractive
│
│   └─ shared_utilities/
│       ├── file_operations.py       File I/O + hashing
│       └── text_cleaning.py         Text normalization
├─ frontend/
│   ├── index.html                  SPA shell
│   ├── css/index.css              Dark glassmorphism UI
│   └── js/app.js                   Frontend logic
├─ alembic/                       DB migrations
├─ tests/                         Pytest suite
├─ scripts/                       Setup scripts
├─ data/ (raw/ processed/ temp/)  Runtime data
├─ models/                        GGUF LLM weights
└─ logs/                         Runtime logs

```